

Access Control Lists

Files and directories have permission sets for the owner of the file, the group associated with the file, and all other users for the system. However, these permission sets have limitations. For example, different permissions cannot be configured for different users. Thus, *Access Control Lists* (ACLs) were implemented.

The Red Hat Enterprise Linux kernel provides ACL support for the ext3 file system and NFS-exported file systems. ACLs are also recognized on ext3 file systems accessed via Samba. In ext4 and xfs it's mounted by default.

Along with support in the kernel, the `acl` package is required to implement ACLs. It contains the utilities used to add, modify, remove, and retrieve ACL information.

The `cp` and `mv` commands copy or move any ACLs associated with files and directories.

Mounting File Systems

Before using ACLs for a file or directory, the partition for the file or directory must be mounted with ACL support. If it is a local ext3 file system, it can be mounted with the following command:

```
mount -t ext3 -o acl device-name partition
```

For example:

```
mount -t ext3 -o acl /dev/VolGroup00/LogVol02 /work
```

Alternatively, if the partition is listed in the `/etc/fstab` file, the entry for the partition can include the `acl` option:

```
LABEL=/work    /work    ext3    acl    1 2
```

If an ext3 file system is accessed via Samba and ACLs have been enabled for it, the ACLs are recognized because Samba has been compiled with the `--with-acl-support` option. No special flags are required when accessing or mounting a Samba share.

Setting Access ACLs

There are two types of ACLs: *access ACLs* and *default ACLs*. An access ACL is the access control list for a specific file or directory. A default ACL can only be associated with a directory; if a file within the directory does not have an access ACL, it uses the rules of the default ACL for the directory. Default ACLs are optional.

ACLs can be configured:

1. Per user
2. Per group
3. Via the effective rights mask
4. For users not in the user group for the file

The `setfacl` utility sets ACLs for files and directories. Use the `-m` option to add or modify the ACL of a file or directory:

```
# setfacl -m rules files
```

Rules (*rules*) must be specified in the following formats. Multiple rules can be specified in the same command if they are separated by commas.

`u:uid:perms`

Sets the access ACL for a user. The user name or UID may be specified. The user may be any valid user on the system.

`g:gid:perms`

Sets the access ACL for a group. The group name or GID may be specified. The group may be any valid group on the system.

`m:perms`

Sets the effective rights mask. The mask is the union of all permissions of the owning group and all of the user and group entries.

`o:perms`

Sets the access ACL for users other than the ones in the group for the file.

Permissions (*perms*) must be a combination of the characters `r`, `w`, and `x` for read, write, and execute.

If a file or directory already has an ACL, and the `setfacl` command is used, the additional rules are added to the existing ACL or the existing rule is modified.

Give read and write permissions

For example, to give read and write permissions to user andrius:

```
# setfacl -m u:andrius:rw /project/somefile
```

To remove all the permissions for a user, group, or others, use the `-x` option and do not specify any permissions:

```
# setfacl -x rules files
```

Remove all permissions

For example, to remove all permissions from the user with UID 500:

```
# setfacl -x u:500 /project/somefile
```

5.3. Setting Default ACLs

To set a default ACL, add `d:` before the rule and specify a directory instead of a file name.

Setting default ACLs

For example, to set the default ACL for the `/share/` directory to read and execute for users not in the user group (an access ACL for an individual file can override it):

```
# setfacl -m d:o:rx /share
```

Retrieving ACLs

To determine the existing ACLs for a file or directory, use the `getfacl` command. In the example below, the `getfacl` is used to determine the existing ACLs for a file.

Retrieving ACLs

```
# getfacl home/john/picture.png
```

The above command returns the following output:

```
# file: home/john/picture.png
# owner: john
# group: john
user::rw-
group::r--
other::r--
```

If a directory with a default ACL is specified, the default ACL is also displayed as illustrated below. For example, `getfacl home/sales/` will display similar output:

```
# file: home/sales/
# owner: john
# group: john
user::rw-
user:barryg:r--
group::r--
mask::r--
other::r--
default:user::rwx
default:user:john:rwx
default:group::r-x
default:mask::rwx
default:other::r-x
```

Archiving File Systems With ACLs

By default, the `dump` command now preserves ACLs during a backup operation. When archiving a file or file system with `tar`, use the `--acls` option to preserve ACLs. Similarly, when using `cp` to copy files with ACLs, include the `--preserve=mode` option to ensure that ACLs are copied across too. In addition, the `-a` option (equivalent to `-dR --preserve=all`) of `cp` also preserves ACLs during a backup along with other information such as timestamps, SELinux contexts, and the like. For more information about `dump`, `tar`, or `cp`, refer to their respective `man` pages.

The `star` utility is similar to the `tar` utility in that it can be used to generate archives of files; however, some of its options are different. Refer to [Mention Below, “Command Line Options for star”](#) for a listing of more commonly used options. For all available options, refer to `man star`. The `star` package is required to use this utility.

Command Line Options for `star`

Option	Description
<code>-c</code>	Creates an archive file.
<code>-n</code>	Do not extract the files; use in conjunction with <code>-x</code> to show what extracting the files does.

Option Description

- `-r` Replaces files in the archive. The files are written to the end of the archive file, replacing any files with the same path and file name.
- `-t` Displays the contents of the archive file.
- `-u` Updates the archive file. The files are written to the end of the archive if they do not exist in the archive, or if the files are newer than the files of the same name in the archive. This option only works if the archive is a file or an unblocked tape that may backspace.
- `-x` Extracts the files from the archive. If used with `-U` and a file in the archive is older than the corresponding file on the file system, the file is not extracted.
- `-help` Displays the most important options.
- `-xhelp` Displays the least important options.
- `-/` Do not strip leading slashes from file names when extracting the files from an archive. By default, they are stripped when files are extracted.
- `-acl` When creating or extracting, archives or restores any ACLs associated with the files and directories.

Compatibility with Older Systems

If an ACL has been set on any file on a given file system, that file system has the `ext_attr` attribute. This attribute can be seen using the following command:

```
# tune2fs -l filesystem-device
```

A file system that has acquired the `ext_attr` attribute can be mounted with older kernels, but those kernels do not enforce any ACLs which have been set.

Versions of the `e2fsck` utility included in version 1.22 and higher of the `e2fsprogs` package (including the versions in Red Hat Enterprise Linux 2.1 and 4) can check a file system with the `ext_attr` attribute. Older versions refuse to check it.